Sovereign AI

at the Edge

Hardware · Models · Training · Inference · Deployment

DGX Spark | Jetson | Gemma 4 | Raspberry Pi | Mac/Ubuntu

GEMMA 4 26B BENCHMARK — tok/s on DGX Spark GB10

TABLE OF CONTENTS

Sources: NVIDIA DGX Spark · Google DeepMind · Jetson AI Lab · Community Benchmarks June 2026

Alto Tech · Bangkok, Thailand · June 2026

CHAPTER 1 — Introduction: What Is Sovereign AI?

Owning Your AI: Data, Models, and Compute

Sovereign AI means complete jurisdictional and operational control over the AI supply chain: the hardware you compute on, the data you train and infer with, the models you run, and the software stack that ties it together. Nothing touches an external server unless you explicitly choose it.

Edge AI takes sovereignty one step further: the compute lives at the source of the data — on a desk, on a factory floor, in a drone, or in a pocket. This delivers sub-millisecond latency, offline operation, and privacy by physical design.

Dimension	Cloud AI	Sovereign Edge AI
Data privacy	Data leaves your perimeter	Data never moves off-device
Latency	50–200 ms round-trip	<5 ms local inference
Uptime dependency	Internet required	Fully offline capable
Cost model	Per-token API fees	One-time hardware investment
Compliance	Complex data residency rules	Guaranteed by physics
Customization	Fine-tuning via API (limited)	Full fine-tuning, any data
Model transparency	Black-box	Open weights, full audit

1.1 The Sovereign AI Stack

A complete sovereign AI stack has five layers. Each layer can be fully on-premise with open-source components available today:

Layer	Component	Open-Source Options
Hardware	Edge compute	DGX Spark, Jetson Orin, Raspberry Pi, Mac, Phone
Models	Open weights	Gemma 4, Llama 4, Qwen 3.5, Mistral, DeepSeek
Runtime	Inference engine	Ollama, vLLM, llama.cpp, LiteRT-LM, TensorRT-LLM
Serving	API gateway	LiteLLM, llama-swap, OpenAI-compatible proxy
Application	Agent framework	NeMo Agent Toolkit, LangChain, OpenClaw

KEY CONCEPT

This guide covers all five layers end-to-end, from booting bare metal to deploying a production-grade agentic system with zero cloud dependency.

1.2 Why Now? The 2026 Inflection Point

Three converging events in 2025-2026 made sovereign edge AI practical for the first time:

- Hardware breakthrough: NVIDIA GB10 Grace Blackwell (DGX Spark) delivers 1 PFLOPS FP4 in a 500W desktop with 128 GB unified memory — enough to run 31B+ models locally.
- Model efficiency: Gemma 4 E2B runs in <1.5 GB RAM on a phone via LiteRT-LM. The 26B MoE activates only 3.8B parameters per token.
- Software maturity: Ollama, vLLM, LiteRT-LM, and llama.cpp now provide one-command deployment across every major platform.

CHAPTER 2 — Edge Hardware Deep Dive

Choosing Your Sovereign Compute Platform

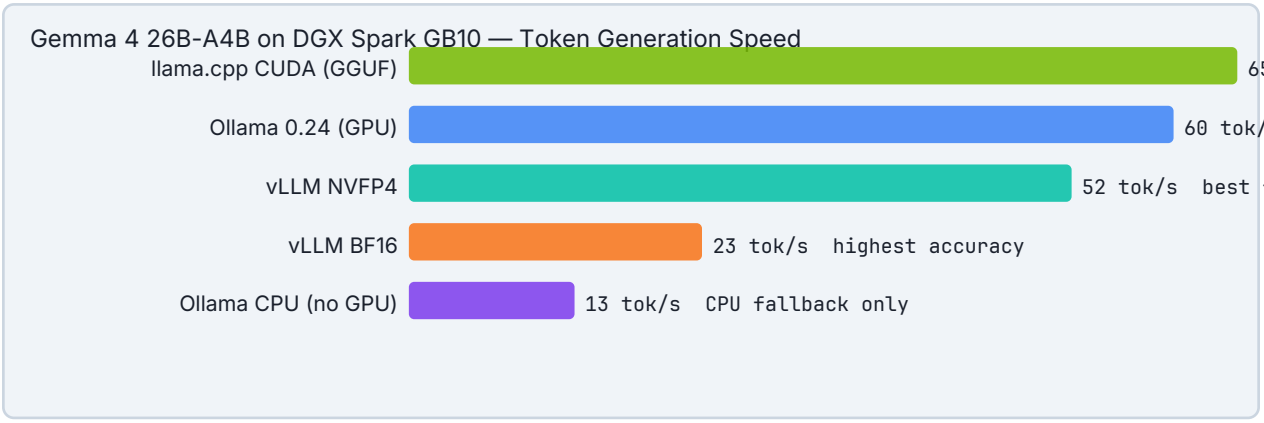
2.1 NVIDIA DGX Spark — The Desktop Supercomputer

The DGX Spark (GB10 Grace Blackwell Superchip) is the most capable personal AI computer ever built. It bridges the gap between consumer GPUs and data-center servers — specifically designed for sovereign, local AI at scale.

Specification	DGX Spark GB10	Notes
CPU	20-core ARM (10× Cortex-X925 + 10× A725)	High-performance + efficiency hybrid
GPU	NVIDIA Blackwell, 5th-gen Tensor Cores	1 PFLOPS FP4 peak
Unified Memory	128 GB LPDDR5x	273 GB/s bandwidth, CPU+GPU share instantly
NVMe Storage	Up to 4 TB self-encrypting	PCIe Gen5
Networking	ConnectX + NVLink-C2C	Cluster two Sparks → 256 GB RAM for 405B
TDP	~500W	Desktop form factor
OS	Ubuntu + NVIDIA stack	JupyterLab, Ollama, vLLM pre-configured
Price (2026)	~$3,000–4,000	Replaces $50K+ cloud spend per year

NVIDIA
Two DGX Sparks can be clustered via NVLink-C2C to share 256 GB unified memory, enabling Llama 4 405B or other frontier models to run completely locally.

DGX Spark — Gemma 4 26B MoE Benchmark (Community, April 2026)



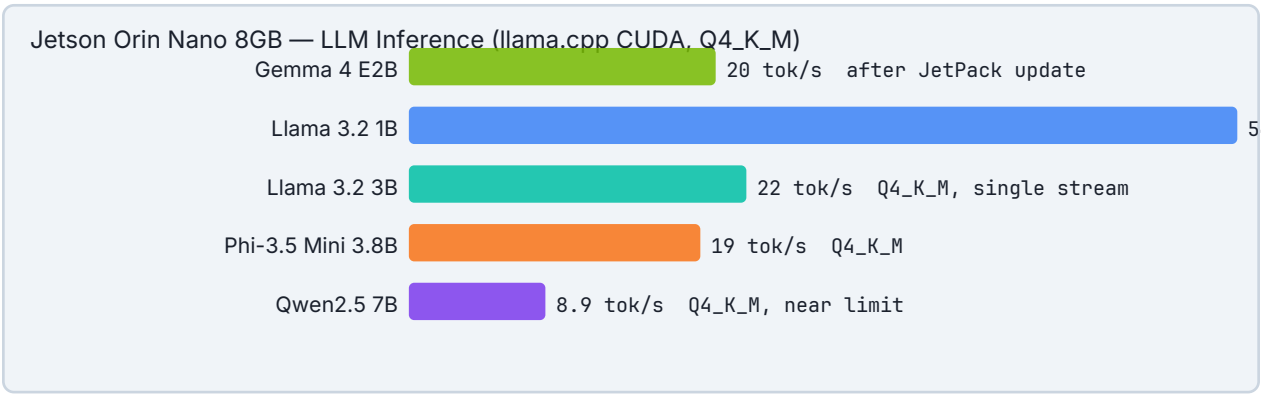
BENCHMARK

vLLM vs Ollama on DGX Spark: vLLM serves Gemma 4 26B at 52 tok/s vs Ollama's 40 tok/s (30% gap) from Marlin NVFP4 kernels, CUDA graphs, and torch.compile fusion. Under 3 concurrent requests, vLLM reaches 114.6 tok/s combined. For single interactive user: prefer Ollama. For 2+ concurrent users: use vLLM.

2.2 NVIDIA Jetson Family

The Jetson platform brings GPU-accelerated edge AI to embedded and robotics applications. From the Orin Nano at $249 to the Thor at the data-center edge, Jetson covers all power-constrained scenarios.

Module	GPU	RAM	AI TOPS	Power	Best For
Jetson Orin Nano 4GB	Ampere 512-core	4 GB	20 TOPS	5–10W	Light vision + small LLMs
Jetson Orin Nano 8GB	Ampere 1024-core	8 GB	40 TOPS	10–15W	Gemma 4 E4B, YOLOv8
Jetson Orin NX 16GB	Ampere 1024-core	16 GB	100 TOPS	15–25W	Gemma 4 26B (quantized)
Jetson AGX Orin 64GB	Ampere 2048-core	64 GB	275 TOPS	40–60W	13B models at speed
Jetson Thor	Blackwell	128 GB	2000 TOPS	~200W	70B models, robotics VLA



KEY CONCEPT

The LLM decode speed gap between Jetson Orin Nano and Raspberry Pi 5 is driven by memory bandwidth: 102 GB/s vs 17 GB/s, predicting the ~4× throughput ratio almost exactly. LLM inference is memory-bandwidth bound, not TOPS bound.

2.3 Raspberry Pi 5

The Raspberry Pi 5 ($80) is the most accessible sovereign AI platform. It cannot match Jetson on performance, but it excels for batch tasks, IoT edge inference, air-gapped deployments, and learning. With the Hailo AI HAT+, vision performance jumps dramatically.

Config	CPU Cores	RAM	AI Accelerator	LLM Speed (1B Q4)	Price
Pi 5 (bare)	4× Cortex-A76	4/8 GB	None (CPU only)	~17 tok/s	$60–80
Pi 5 + AI Kit	4× Cortex-A76	8 GB	Hailo-8L (13 TOPS)	LLM: CPU only	$90
Pi 5 + AI HAT+	4× Cortex-A76	8 GB	Hailo-8 (26 TOPS)	Vision: 105 FPS	$130
Pi 5 + Hailo-10H	4× Cortex-A76	8 GB	Hailo-10H (40+ TOPS)	NPU-accel LLM	$150

⚠ WARNING

Critical: Hailo accelerators do NOT accelerate LLMs — they are for vision workloads only. For LLM inference on Pi 5, you are always on CPU. Use models ≤1.5B for practical interactive speeds (17 tok/s on 1B). Gemma 4 E2B via LiteRT-LM achieves 7.6 decode tok/s and 133 prefill tok/s with optimized runtime.

2.4 Apple Silicon Mac

Apple Silicon Macs are exceptional sovereign AI platforms. The unified memory architecture means GPU and CPU share RAM — a 64 GB M2 Ultra has 64 GB "VRAM". MLX (Apple's ML framework) and Metal GPU backend give near-NVIDIA speeds for LLM inference.

Mac	Unified RAM	GPU Cores	Recommended Model	Speed (Q4_K_M)
MacBook Air M4 16 GB	16 GB	10-core	Gemma 4 26B MoE	~15 tok/s
MacBook Pro M4 Pro 24 GB	24 GB	20-core	Gemma 4 26B MoE	~22 tok/s
MacBook Pro M4 Max 48 GB	48 GB	40-core	Gemma 4 31B Dense	~25 tok/s
Mac Studio M2 Ultra 64 GB	64 GB	60-core	Gemma 4 31B Dense (Q8)	~20 tok/s
Mac Studio M4 Ultra 192 GB	192 GB	80-core	Llama 4 405B MoE	~30 tok/s

GOOGLE

Ollama 0.23.1+ added Gemma 4 MTP (Multi-Token Prediction) speculative decoding on Mac via the MLX runner — achieving a 2× speed increase on Gemma 4 31B coding tasks on Apple Silicon. The drafter reuses the target model's KV cache for zero overhead.

2.5 Phones & Mobile Devices

Device/Platform	Runtime	Best Model	Performance	Memory

Android (AICore)	LiteRT-LM	Gemma 4 E2B	Varies by chipset	<1.5 GB
Android (Qualcomm NPU)	LiteRT-LM	Gemma 4 E2B	31 decode tok/s	<1.5 GB
iPhone (CoreML)	CoreML-LLM	Gemma 4 E2B	11 tok/s on iPhone 16	250 MB
Samsung Galaxy S25	LiteRT-LM + NPU	Gemma 4 E4B	~25 tok/s	<3 GB
Arduino VENTUNO Q	LiteRT-LM	Gemma 4 E2B	31 tok/s (Dragonwing NPU)	1.5 GB

GOOGLE

Google AI Edge Gallery (iOS + Android) ships with Agent Skills — the first multi-step autonomous agentic workflows running entirely on-device powered by Gemma 4. Processes 4,000 input tokens across 2 skills in under 3 seconds on capable devices.

2.6 Hardware Decision Matrix

Use Case	Recommended Platform	Min Model	Why
Production local assistant	DGX Spark	Gemma 4 31B	Max quality, multi-user, fine-tune
Robotics / VLA	Jetson Orin / Thor	Gemma 4 E4B	Low power, GPU vision + LLM together
IoT / Air-gapped cluster	Raspberry Pi 5 ×N	Gemma 4 E2B	Ultra-low cost, cluster horizontally
Developer workstation	Mac M4 Pro/Max	Gemma 4 26B MoE	Best UX, silent, battery life
Mobile agent	Android/iOS phone	Gemma 4 E2B	Always with user, offline capable
Smart building / HVAC edge	DGX Spark + IoT GW	Gemma 4 31B	High-context MAS on premise

CHAPTER 3 — Gemma 4 Family & Benchmarks

Google DeepMind's Frontier Edge Model (April 2026)

Gemma 4 launched April 2, 2026 under Apache 2.0 — fully open, commercially usable, no restrictions. The family spans from sub-1.5 GB mobile models to a 31B dense model that hits #3 on the Arena AI leaderboard, beating models 20× its size. Every variant supports 140+ languages and native function-calling for agentic workflows.

3.1 Model Variants

Variant	Params	Active/Token	Min VRAM (Q4)	Context	Primary Target
Gemma 4 E2B	2.3B	2.3B (dense)	~1.5 GB	128K	Phone, RPi, IoT
Gemma 4 E4B	4.5B	4.5B (dense)	~3 GB	128K	Phone, Jetson Nano, tablet
Gemma 4 26B-A4B (MoE)	26B	3.8B (MoE)	~14 GB	256K	Laptop, Mac, Jetson NX
Gemma 4 31B (Dense)	31B	31B (dense)	~18 GB	256K	DGX Spark, workstation

KEY CONCEPT

The 26B MoE is the sweet spot: activates only 3.8B parameters per token (6× faster decode than 31B dense) while delivering GPQA Diamond 82.3% vs 84.3% for the full 31B. Fits on a single 16 GB GPU at Q4. The preferred choice for developers with a Mac M4 Pro or Jetson Orin NX.

3.2 Benchmark Showdown

Model	MMLU Pro	AIME 2026	LiveCodeBench v6	GPQA Diamond	Codeforces ELO
Gemma 4 31B	85.2%	89.2%	80.0%	84.3%	2150
Gemma 4 26B MoE	82.6%	88.3%	77.1%	82.3%	1718
Gemma 4 E4B	69.4%	42.5%	52.0%	—	940
Gemma 4 E2B	60.0%	37.5%	44.0%	—	633
Gemma 3 27B (prev gen)	67.6%	20.8%	29.1%	42.4%	110

Llama 3.3 70B	~75%	~40%	~62%	~60%	—

BENCHMARK

Gemma 4 31B scores 89.2% on AIME 2026 (math olympiad) vs Gemma 3's 20.8% — a 4× improvement in one generation. On LiveCodeBench v6 (coding), the 26B MoE at 77.1% beats Llama 3.3 70B while using a fraction of the memory.

3.3 Architecture: Dense vs. Mixture-of-Experts

Gemma 4 introduces two architectural families. Understanding the difference is critical for hardware selection:

Architecture	How It Works	Speed	Memory	Quality	Best Use
Dense (31B)	All 31B params active per token	Slower	18 GB Q4	Highest	Max quality tasks
MoE (26B-A4B)	3.8B active per token, 26B total	6× faster decode	14 GB Q4	Near-31B	Production serving

3.4 Key Features for Sovereign Edge Agents

- 140+ languages with single-model multilingual support — no language-specific fine-tuning needed
- 128K–256K dynamic context — entire codebases, long documents, or multi-session history in one call
- Native function calling — structured JSON tool-call output, constrained decoding for reliability
- Multimodal (E2B/E4B) — vision and audio embeddings; process images and audio on-device
- Apache 2.0 license — commercial use, fine-tuning, and redistribution without restrictions
- LiteRT-LM optimized — <1.5 GB on mobile, constrained decoding for reliable agents, 128K dynamic context
- AICore integration — system-wide Android access via Android's built-in Gemma 4 through AICore Developer Preview

3.5 Performance Across All Platforms

Platform	Model	Runtime	Speed	Notes
DGX Spark GB10	Gemma 4 26B MoE	llama.cpp CUDA	65 tok/s	Speed king on Spark
DGX Spark GB10	Gemma 4 26B MoE	Ollama 0.24	60 tok/s	Easiest setup
DGX Spark GB10	Gemma 4 26B MoE	vLLM NVFP4	52 tok/s	Best multi-user

RTX 4090 (24 GB)	Gemma 4 31B Dense	Ollama Q4_K_M	~35 tok/s	Consumer GPU
RTX 4070 Ti (16 GB)	Gemma 4 26B MoE	Ollama Q4_K_M	~30 tok/s	16 GB GPU
Mac M3 Pro 18 GB	Gemma 4 26B MoE	Ollama Q4_K_M	~15 tok/s	Unified memory
Jetson Orin Nano 8GB	Gemma 4 E4B	llama.cpp CUDA	~20 tok/s	Embedded edge
Raspberry Pi 5	Gemma 4 E2B	LiteRT-LM	7.6 tok/s	133 prefill tok/s
Qualcomm NPU phone	Gemma 4 E2B	LiteRT-LM	31 tok/s	NPU accelerated
iPhone 16	Gemma 4 E2B	CoreML-LLM	11 tok/s	250 MB RAM, 2W

CHAPTER 4 — Model Preparation & NVFP4 Quantization INTERMEDIATE

Optimizing Models for Edge Deployment

Raw model weights from Hugging Face are in BF16 or FP32 — too large for most edge devices. Quantization compresses models to 4-bit or 8-bit precision with minimal quality loss. NVIDIA's NVFP4 format is purpose-built for Blackwell Tensor Cores, delivering near-8-bit accuracy at 4-bit size and speed.

4.1 Quantization Formats Compared

Format	Bits	Relative Size	Quality Loss	Best Runtime	Best Hardware
BF16	16	1.0×	None (baseline)	vLLM, Ollama	Any modern GPU
FP8	8	0.5×	<1%	vLLM, TensorRT-LLM	Hopper/Blackwell
NVFP4 (Blackwell)	4	0.25×	<5%	vLLM + modelopt	Blackwell only
Q8_0 (GGUF)	8	~0.5×	<1%	llama.cpp, Ollama	Any GPU/CPU
Q4_K_M (GGUF)	4	~0.25×	<5%	llama.cpp, Ollama	Any GPU/CPU
Q4 (LiteRT)	4	~0.25×	<5%	LiteRT-LM	Mobile/RPi
2-bit (LiteRT)	2	~0.12×	Moderate	LiteRT-LM	Ultra-low-memory

NVIDIA

NVFP4 is Blackwell-exclusive. It exploits 5th-generation Tensor Cores that natively process 4-bit float (FP4) with block scaling. On a DGX Spark GB10, NVFP4 provides 1.8–2.3× tokens/sec improvement over FP8 with <5% accuracy drop. For non-Blackwell hardware, use Q4_K_M (GGUF) for the same quality/size tradeoff.

4.2 NVFP4 Quantization with NVIDIA Model Optimizer

NVIDIA Model Optimizer (formerly TensorRT Model Optimizer) is the official tool for generating NVFP4 checkpoints from Hugging Face BF16 weights. The workflow takes 30–60 minutes on a DGX Spark for a 31B model.

```
# Step 1: Download model from Hugging Face
pip install huggingface_hub
huggingface-cli download google/gemma-4-31b-it \
--local-dir ./gemma4-31b \
```

```
--exclude "*.msgpack" "*.h5"
# Verify download
ls -lh ./gemma4-31b/*.safetensors
```

```
# Step 2: Install NVIDIA Model Optimizer
pip install nvidia-modelopt[torch] --extra-index-url \
https://pypi.nvidia.com
# Step 3: Quantize to NVFP4
# calibration dataset = wikitext (representative text for scale estimation)
python -m modelopt.torch.quantization.quantize \
--model_name_or_path ./gemma4-31b \
--qformat nvfp4 \
--output_dir ./gemma4-31b-nvfp4 \
--calib_dataset wikitext \
--calib_size 512 \
--tp_size 1
# Output: ./gemma4-31b-nvfp4/ with quantized weights + scaling factors
# Expected size: ~8 GB (vs ~62 GB BF16)
```

```
# Step 4: Serve NVFP4 model with vLLM on DGX Spark
docker run -d --gpus all \
-p 8000:8000 \
--ipc=host \
--shm-size 64g \
-v ./gemma4-31b-nvfp4:/models \
vllm/vllm-openai:latest \
--model /models \
--quantization modelopt \
--kv-cache-dtype fp8 \
--max-model-len 131072 \
--gpu-memory-utilization 0.88 \
--moe-backend marlin \
--enable-auto-tool-choice \
--reasoning-parser gemma4 \
--served-model-name gemma-4-31b
# Test endpoint
curl http://localhost:8000/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{"model":"gemma-4-31b","messages":[{"role":"user","content":"Hello"}]}'
```

4.3 GGUF Quantization for llama.cpp / Ollama

For non-Blackwell hardware (Mac, Raspberry Pi, RTX GPUs), GGUF with Q4_K_M is the recommended quantization. llama.cpp handles both conversion and quantization.

```
# Clone llama.cpp and build with CUDA (for GPU systems)
git clone https://github.com/ggerganov/llama.cpp
```

```
cd llama.cpp
cmake -B build -DGGML_CUDA=ON && cmake --build build --config Release -j$(nproc)
# Convert HuggingFace model → GGUF (BF16)
python convert_hf_to_gguf.py ./gemma4-26b --outfile gemma4-26b-bf16.gguf
# Quantize BF16 GGUF → Q4_K_M (recommended)
./build/bin/llama-quantize gemma4-26b-bf16.gguf gemma4-26b-q4km.gguf Q4_K_M
# Serve as OpenAI-compatible API
./build/bin/llama-server \
--model gemma4-26b-q4km.gguf \
--ctx-size 32768 \
--n-gpu-layers 99 \
--host 0.0.0.0 \
--port 8080
```

TIP

Pre-quantized GGUF models are available on Hugging Face from community repos (bartowski, TheBloke, etc.). For Gemma 4: search "gemma-4-26b GGUF Q4_K_M". This saves hours of conversion time.

4.4 LiteRT-LM Quantization for Mobile/Pi

```
# Install LiteRT-LM CLI
pip install litert-lm
# Run Gemma 4 E2B directly on macOS / Linux / Raspberry Pi
litert-lm run gemma4-e2b
# With Metal GPU backend on Mac
litert-lm run gemma4-e2b --device metal
# Tool calling mode (powers Agent Skills)
litert-lm run gemma4-e2b --enable-tool-calling
# Convert Hugging Face model to LiteRT flatbuffer format
litert-lm convert \
--hf_model google/gemma-4-e2b-it \
--quant_bits 4 \
--output gemma4-e2b-q4.litert
```

CHAPTER 5 — Fine-Tuning on Edge (NeMo AutoModel) ADVANCED

Domain Adaptation Without the Cloud

Fine-tuning lets you adapt a base model to your exact domain: your HVAC system's fault codes, your company's internal documentation, your local language dialect. On a DGX Spark, fine-tuning a 31B model via LoRA takes 45–90 minutes — shorter than a typical cloud training job at a fraction of the cost.

5.1 NeMo AutoModel — The Recommended Framework

NVIDIA NeMo AutoModel combines native PyTorch ease-of-use with DGX Spark-optimized performance. Key advantages:

- Day-0 fine-tuning: starts from Hugging Face checkpoints with no conversion needed
- Memory-efficient LoRA: fine-tune 31B models in 128 GB unified memory
- FP8 precision: faster and more memory-efficient on Blackwell
- ARM64 optimized: compiled kernels specifically for GB10 architecture
- Distributed training: scales from single GPU to clustered Sparks automatically

5.2 NeMo AutoModel Setup on DGX Spark

```
# Prerequisites: SSH into your DGX Spark
# Hardware: DGX Spark GB10, CUDA 12.0+, Python 3.10+, 32 GB RAM minimum
# Step 1: Pull NeMo Automodel Docker image (recommended)
docker pull nvcr.io/nvidia/nemo:nemo-automodel-latest
# Step 2: Clone the Automodel recipes
git clone https://github.com/NVIDIA-NeMo/Automodel
cd Automodel
# Step 3: Launch container with GPU access + model volume
docker run --gpus all --ipc=host --shm-size 64g \
-v $(pwd):/workspace \
-v ~/.cache/huggingface:/root/.cache/huggingface \
-w /workspace \
-it nvcr.io/nvidia/nemo:nemo-automodel-latest bash
```

```
# Step 4: Prepare your training dataset (SFT format)
# Format: JSONL with "input" and "output" fields
cat > dataset.jsonl << EOF
{"input": "What is the chiller fault CH-F03?", "output": "CH-F03 is condenser approach temperature high..."}
{"input": "Recommend action for AHU-F02", "output": "AHU-F02 filter differential pressure high..."}
```

```
EOF
# Step 5: Configure fine-tuning YAML for Gemma 4 31B
cat > finetune_gemma4.yaml << EOF
model:
pretrained_model_name_or_path: google/gemma-4-31b-it
precision: bf16 # or fp8 for Blackwell
training:
method: lora # parameter-efficient
lora_rank: 16
lora_alpha: 32
target_modules: [q_proj, v_proj, k_proj, o_proj]
data:
train_file: dataset.jsonl
max_seq_length: 4096
batch_size: 2
gradient_accumulation_steps: 8
optimizer:
lr: 2.0e-4
epochs: 3
output:
checkpoint_dir: ./checkpoints/gemma4-hvac-lora
EOF
```

```
# Step 6: Run fine-tuning
python -m nemo.collections.llm.train \
--config finetune_gemma4.yaml
# Expected output (DGX Spark, 31B LoRA):
# Loading model: ~8 min
# Training epoch 1/3: ~20 min
# Training epoch 2/3: ~18 min
# Training epoch 3/3: ~17 min
# Saving checkpoint: ~5 min
# Total: ~68 min
# Step 7: Merge LoRA weights back into base model
python -m nemo.collections.llm.merge_lora \
--base_model google/gemma-4-31b-it \
--lora_checkpoint ./checkpoints/gemma4-hvac-lora \
--output_dir ./gemma4-hvac-merged
# Step 8: Serve the fine-tuned model with Ollama
# Create Modelfile
cat > Modelfile << EOF
FROM ./gemma4-hvac-merged
SYSTEM "You are an expert HVAC fault diagnosis assistant..."
EOF
ollama create gemma4-hvac -f Modelfile
ollama run gemma4-hvac
```

5.3 Fine-Tuning Best Practices

Setting	Recommended Value	Why
LoRA rank	16–64	Higher = more capacity, more memory
Learning rate	1e-4 to 3e-4	Too high = catastrophic forgetting
Batch size	2–4 (gradient acc. ×8)	Memory vs. gradient quality
Max seq length	2048–8192	Match your actual use-case context
Precision	BF16 or FP8 on Blackwell	FP8 saves 50% memory vs BF16
Dataset size	500–5000 examples	More doesn't always help; quality matters
Epochs	2–5	Monitor eval loss; stop early if overfitting

CHAPTER 6 — Inference & Serving (vLLM · Ollama · LiteLLM INTERMEDIATE

Building Your Local AI API Endpoint

Once your model is ready, you need a serving layer that any application can call via the OpenAI-compatible REST API. Three runtimes cover all scenarios: Ollama for simplicity, vLLM for production throughput, LiteLLM as a unified proxy that routes to any backend.

6.1 Ollama — Quickest Path to Running

Ollama provides one-command model management and serving with an OpenAI-compatible endpoint. It handles model download, quantization selection, and GPU offloading automatically.

```
# Install Ollama (Linux / macOS / DGX Spark)
curl -fsSL https://ollama.com/install.sh | sh
# Run Gemma 4 models (auto-downloads on first run)
ollama run gemma4:2b # Gemma 4 E2B — for Raspberry Pi / low RAM
ollama run gemma4:4b # Gemma 4 E4B — for phones / Jetson
ollama run gemma4:26b # Gemma 4 26B MoE — RECOMMENDED for DGX/Mac
ollama run gemma4:31b # Gemma 4 31B Dense — max quality on DGX Spark
# Verify GPU is being used (must show GPU utilization)
nvidia-smi # should show ~80-95% GPU memory used
# If Ollama version < 0.24 on DGX Spark, GPU may not be detected:
ollama --version # check version
curl -fsSL https://ollama.com/install.sh | sh # upgrade
# Ollama REST API (OpenAI-compatible)
curl http://localhost:11434/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{"model":"gemma4:26b","messages":[
{"role":"user","content":"What is NVFP4?"}]}'
```

⚠ WARNING

Critical for DGX Spark: Ollama versions before 0.24 were missing sm_121 CUDA libraries for Blackwell, causing CPU-only inference at 13 tok/s instead of GPU at 60 tok/s. Always use Ollama 0.24+ on DGX Spark.

6.2 vLLM — Production-Grade High-Throughput Serving

vLLM is the industry standard for production LLM serving. It achieves higher throughput than Ollama under multi-user load through PagedAttention, CUDA graphs, and compiled kernels. Use vLLM whenever you have 2+ concurrent users or need OpenAI-compatible streaming.

```
# Option A: Docker (recommended for production)
docker run -d --gpus all \
--name vllm-gemma4 \
-p 8000:8000 \
--ipc=host \
--shm-size 64g \
-v ~/.cache/huggingface:/root/.cache/huggingface \
vllm/vllm-openai:latest \
--model google/gemma-4-27b-it \
--max-model-len 32768 \
--gpu-memory-utilization 0.90 \
--enable-auto-tool-choice \
--tool-call-parser hermes \
--served-model-name gemma-4
# For NVFP4 quantized model on Blackwell
docker run -d --gpus all -p 8000:8000 --ipc=host --shm-size 64g \
-v ./gemma4-31b-nvfp4:/models \
vllm/vllm-openai:latest \
--model /models \
--quantization modelopt \
--kv-cache-dtype fp8 \
--moe-backend marlin \
--enable-auto-tool-choice
# Option B: pip install (for development)
pip install vllm
vllm serve google/gemma-4-27b-it \
--port 8000 \
--gpu-memory-utilization 0.90
```

```
# Python client (OpenAI SDK compatible)
from openai import OpenAI
client = OpenAI(base_url="http://localhost:8000/v1", api_key="not-needed")
response = client.chat.completions.create(
model="gemma-4",
messages=[
{"role": "system", "content": "You are a sovereign AI assistant."},
{"role": "user", "content": "Explain NVFP4 quantization."}
],
temperature=0.7,
max_tokens=512,
stream=True
)
for chunk in response:
print(chunk.choices[0].delta.content or "", end="", flush=True)
```

6.3 LiteLLM + llama-swap — The Production Stack

LiteLLM is a unified proxy that routes requests to any backend using the OpenAI API format. Combined with llama-swap (which hot-swaps models in/out of GPU memory), you get a complete production stack: any app calls one LiteLLM endpoint, which routes to the optimal backend and manages model memory automatically.

```
# Full production stack: LiteLLM → llama-swap → {vLLM, Ollama, llama.cpp}
# Source: NVIDIA DGX Spark Community Forum (2026-04-23)
# docker-compose.yml
# ---
version: "3.8"
services:
llama-swap:
image: ghcr.io/mostlygeek/llama-swap:latest
volumes:
- ./swap-config.yaml:/app/config.yaml
- ~/.cache/huggingface:/models
ports: ["8001:8001"]
restart: always
litellm:
image: ghcr.io/berriai/litellm:main-latest
volumes:
- ./litellm-config.yaml:/app/config.yaml
ports: ["8000:8000"]
depends_on: [llama-swap]
restart: always
command: ["--config", "/app/config.yaml"]
```

```
# litellm-config.yaml — route all models through llama-swap
model_list:
- model_name: gemma-4-fast
litellm_params:
model: openai/gemma4-26b-moe
api_base: http://llama-swap:8001/v1
- model_name: gemma-4-quality
litellm_params:
model: openai/gemma4-31b-dense
api_base: http://llama-swap:8001/v1
- model_name: embedding
litellm_params:
model: openai/nomic-embed-text
api_base: http://llama-swap:8001/v1
litellm_settings:
drop_params: true
set_verbose: false
```

```
# swap-config.yaml — llama-swap model memory management
models:
```

```
gemma4-26b-moe:
cmd: >
llama-server
--model /models/gemma4-26b-q4km.gguf
--n-gpu-layers 99
--ctx-size 32768
--port 9001
port: 9001
ttl: 300 # unload after 5 min idle
gemma4-31b-dense:
cmd: >
llama-server
--model /models/gemma4-31b-q4km.gguf
--n-gpu-layers 99
--ctx-size 16384
--port 9002
port: 9002
ttl: 180
# Start the stack
docker compose up -d
# All apps point to: http://localhost:8000/v1
```

6.4 When to Use Each Runtime

Scenario	Runtime	Reason
Single developer, quick start	Ollama	One command, auto GPU, model library
Production, 2+ concurrent users	vLLM	PagedAttention, 30% more throughput
Multiple models, one endpoint	LiteLLM + llama-swap	Route + hot-swap models
Raspberry Pi / CPU-only	llama.cpp (server mode)	No GPU required, GGUF native
Mobile / Raspberry Pi	LiteRT-LM	Optimized for mobile/IoT
Mac with Apple Silicon	Ollama (MLX runner) or LiteRT-LM	Metal GPU + MTP speculative
Blackwell NVFP4 production	vLLM + modelopt	Native NVFP4 kernel support

CHAPTER 7 — Platform-Specific Deployment Tutorials

Copy-Paste Deployment Guides for Every Platform

7.1 DGX Spark — Full Stack Deployment

BEGINNER

Initial DGX Spark Setup

```
# After initial DGX Spark boot, update system
sudo apt update && sudo apt upgrade -y
# Verify GPU is detected
nvidia-smi
# Expected: GB10 Superchip, 128 GB VRAM
# Install Ollama (0.24+)
curl -fsSL https://ollama.com/install.sh | sh
systemctl --user enable ollama && systemctl --user start ollama
# Run first model — Gemma 4 26B MoE (best for DGX Spark)
ollama run gemma4:26b
# First run downloads ~14 GB. Subsequent runs are instant.
# Install Open WebUI for browser-based chat
docker run -d -p 3000:8080 \
-e OLLAMA_BASE_URL=http://host.docker.internal:11434 \
-v open-webui:/app/backend/data \
--name open-webui \
ghcr.io/open-webui/open-webui:main
# Open: http://localhost:3000
```

```
# vLLM quick start on DGX Spark
pip install vllm
# Serve Gemma 4 26B MoE with OpenAI-compatible endpoint
vllm serve google/gemma-4-27b-it \
--port 8000 \
--gpu-memory-utilization 0.90 \
--max-model-len 32768 \
--enable-auto-tool-choice
# Benchmark: should achieve 52 tok/s (NVFP4) or 60 tok/s (Ollama)
python -c "
from openai import OpenAI; import time
c = OpenAI(base_url="http://localhost:8000/v1", api_key="x")
start = time.time()
r = c.chat.completions.create(model="google/gemma-4-27b-it",
```

```
messages=[{"role":"user","content":"Count from 1 to 100"}],
max_tokens=200)
elapsed = time.time()-start
tokens = r.usage.completion_tokens
print(f"{tokens/elapsed:.1f} tok/s")
"
```

7.2 NVIDIA Jetson — Edge Deployment

INTERMEDIATE

```
# Prerequisites: JetPack 6.x installed on Jetson Orin
# Check JetPack version
cat /etc/nv_tegra_release
# Install Jetson AI Lab containers
git clone https://github.com/dusty-nv/jetson-containers
cd jetson-containers && ./install.sh
# Run Ollama container on Jetson (GPU-accelerated)
./run.sh $(./autotag ollama)
# Inside container:
ollama run gemma4:2b # E2B for Orin Nano 4GB
ollama run gemma4:4b # E4B for Orin Nano 8GB
# Or use llama.cpp with CUDA backend for max speed
./run.sh $(./autotag llama.cpp)
llama-server \
--model gemma4-e4b-q4km.gguf \
--n-gpu-layers 99 \
--ctx-size 8192 \
--host 0.0.0.0 --port 8080
# Expected speeds (Jetson Orin Nano 8GB, after JetPack 6.x update)
# Gemma 4 E2B: ~20 tok/s (10x improvement over pre-update ~2 tok/s)
# Gemma 4 E4B: ~12 tok/s
# Llama 3.2 1B: ~54 tok/s
```

⚠ WARNING

Before JetPack 6.x update, Gemma 4 on Jetson Orin ran at only ~2 tok/s. After the update, it jumps to ~20 tok/s — a 10× improvement from CUDA/TensorRT optimizations. Always run sudo apt update && sudo apt upgrade before benchmarking.

7.3 Raspberry Pi 5 — Ultra-Low-Cost Sovereign AI

BEGINNER

```
# Raspberry Pi 5 setup (8 GB model recommended)
# Step 1: Update system
sudo apt update && sudo apt full-upgrade -y
# Step 2: Install Ollama
curl -fsSL https://ollama.com/install.sh | sh
# Step 3: Run Gemma 4 E2B (optimized for Pi)
ollama run gemma4:2b
```

```
# Expected: ~5 tok/s decode, ~133 tok/s prefill via LiteRT-LM
# Step 4: Install LiteRT-LM for better Pi performance
pip install litert-lm
litert-lm run gemma4-e2b
# LiteRT-LM achieves 7.6 decode tok/s vs ~5 tok/s with Ollama on Pi 5
# Step 5: Run as always-on API server
nohup ollama serve &
# Bind to local network so other Pi nodes can access
OLLAMA_HOST=0.0.0.0:11434 nohup ollama serve &
# Pi 5 performance guide:
# Gemma 4 E2B (Q4): ~5 tok/s — use for interactive chat
# Gemma 4 E2B LiteRT: ~7.6 tok/s — preferred runtime
# Llama 3.2 1B (Q4): ~17 tok/s — fastest for simple tasks
# Qwen2.5 7B (Q4): ~2.3 tok/s — too slow for interactive use
```

TIP

Raspberry Pi Cluster: For higher throughput, run multiple Pi 5 nodes with different small models and use LiteLLM to load-balance across them. A cluster of 4× Pi 5 nodes costs ~$400 and can serve 4 simultaneous users at 5 tok/s each.

7.4 Apple Silicon Mac — GPU-Optimized Local AI

BEGINNER

```
# macOS Apple Silicon — Ollama with Metal GPU backend
# Install Ollama for Mac
brew install ollama
# Or download from https://ollama.com
# Start Ollama service
ollama serve &
# Run models (Mac 16 GB: 26B MoE fits perfectly)
ollama run gemma4:26b # ~15 tok/s on M3 Pro 18 GB
ollama run gemma4:31b # ~20 tok/s on M2 Ultra 64 GB
# Enable MTP speculative decoding (Ollama 0.23.1+ on Mac)
# This gives 2× speed on coding tasks!
# Automatic when using MLX runner — just update Ollama
# LiteRT-LM with Metal backend
pip install litert-lm
litert-lm run gemma4-e2b --device metal
# Install LM Studio (GUI) for non-developers
# Download: https://lmstudio.ai
# Supports: Gemma 4, Llama, Qwen, Mistral with one-click download
# Python OpenAI client (works with Ollama on Mac)
pip install openai
python -c "
from openai import OpenAI
c = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")
r = c.chat.completions.create(model="gemma4:26b",
```

```
messages=[{"role":"user","content":"Hello!"}])
print(r.choices[0].message.content)
"
```

7.5 Phone (Android / iOS) — Pocket Sovereign AI

BEGINNER

```
# Android — Google AI Edge Gallery (easiest path)
# 1. Open Google Play Store
# 2. Search: "Google AI Edge Gallery"
# 3. Install and open
# 4. Go to "Agent Skills" tab
# 5. Gemma 4 E2B downloads automatically (~1.5 GB)
# 6. Try: Wikipedia search, flashcard generation, photo-to-music
# Android — AICore Developer Preview (system-wide access)
# Allows your app to access built-in Gemma 4 without extra download
# Android — Python via adb + LiteRT-LM (developer mode)
# Requires: USB debugging enabled, Python via Termux
pkg install python
pip install litert-lm
litert-lm run gemma4-e2b
# Expected: ~31 tok/s on Qualcomm Snapdragon 8 Gen 3 with NPU
# iOS — CoreML-LLM (Gemma 4 E2B)
# Install: TestFlight or direct download
# Performance: 11 tok/s on iPhone 16, 250 MB RAM, 2W power draw
# Context: 128K window supported
# Agentic workflow on phone: 4,000 input tokens across 2 skills < 3 sec
# (Qualcomm Dragonwing IQ8 NPU, LiteRT-LM)
```

7.6 Ubuntu PC / Server — GPU Workstation

BEGINNER

```
# Ubuntu 22.04+ with NVIDIA GPU (RTX 3090 or better)
# Install CUDA 12.x (if not already installed)
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64/cuda-keyring_1.1-1_all.deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt update && sudo apt install cuda-toolkit-12-4
# Install Ollama
curl -fsSL https://ollama.com/install.sh | sh
# RTX 4090 (24 GB): run 31B Dense at ~35 tok/s
ollama run gemma4:31b
# RTX 4070 Ti (16 GB): run 26B MoE at ~30 tok/s
ollama run gemma4:26b
# RTX 3060 (12 GB): run E4B at ~45 tok/s
ollama run gemma4:4b
# Enable Ollama to listen on network (for LiteLLM proxy)
sudo systemctl edit ollama --force << EOF
```

```
[Service]
Environment="OLLAMA_HOST=0.0.0.0:11434"
EOF
sudo systemctl restart ollama
```

CHAPTER 8 — NeMo Agent Toolkit & NemoClaw Sandbox INTERMEDIATE

Building Sovereign Agents with NVIDIA NeMo

8.1 NeMo Agent Toolkit (nvidia-nat)

The NVIDIA Agent Intelligence Toolkit (nvidia-nat) is a lightweight, flexible Python framework for building multi-agent systems. It works alongside any existing agentic framework (LangChain, LlamaIndex, etc.) and is not tied to any specific backend or memory store.

```
# Install NeMo Agent Toolkit
pip install nvidia-nat
# With LangChain integration
pip install "nvidia-nat[langchain]"
# Hello World — ReAct agent with Wikipedia tool
# 1. Create workflow.yml
cat > workflow.yml << EOF
functions:
wikipedia_search:
_type: wiki_search
max_results: 2
llms:
local_gemma4:
_type: openai
model_name: gemma-4
api_base: http://localhost:8000/v1
api_key: not-needed
temperature: 0.0
workflow:
_type: react_agent
tool_names: [wikipedia_search]
llm_name: local_gemma4
verbose: true
parse_agent_response_max_retries: 3
EOF
# 2. Run the agent
nat run --config_file workflow.yml --input "What is the DGX Spark GB10?"
```

8.2 Key Features

Feature	Details
Framework-agnostic	Works with LangChain, LlamaIndex, CrewAI, or plain Python

MCP compatible	Acts as MCP client or publishes tools as MCP server
A2A compatible	Connects to remote A2A agents or publishes as discoverable A2A agent
Profiling	Tracks tokens, timings, bottlenecks per agent/tool
Observability	LangSmith, Phoenix, Weave, Langfuse, OpenTelemetry
UI chat	Built-in web UI to test and debug agents interactively
Evaluation	Built-in eval tools to validate accuracy of agentic workflows
Composability	Build once, reuse agents/tools across multiple workflows

8.3 NemoClaw — Secure Agent Sandbox

NemoClaw is an open-source reference stack for running OpenClaw always-on agents on DGX Spark with strong security isolation. It uses OpenShell — a sandboxed execution environment that isolates filesystem, network, process, and inference paths.

```
# One-command NemoClaw installation
curl -fsSL https://raw.githubusercontent.com/NVIDIA/nemoclaw/main/nemoclaw.sh | bash
# The installer handles: Node.js + OpenShell runtime + NemoClaw CLI
# Onboard: create your first sandboxed agent
nemoclaw onboard --fresh --gpu
# The onboard wizard will ask:
# 1. Agent name and personality
# 2. Policy tier (conservative / balanced / permissive)
# 3. Network presets (local-only / allow-search / full)
# 4. Messaging channel (Web UI / Telegram / Discord / Slack)
# 5. Optional: enable Brave Search for web access
# Connect Telegram (optional)
# nemoclaw will prompt for a Telegram bot token
# cloudflared creates a public webhook URL automatically
# Access your agent
nemoclaw start # starts the agent + Web UI on http://localhost:7860
nemoclaw tui # terminal TUI mode
```

OpenShell Isolation Layer	What It Blocks
Filesystem	Reads/writes outside explicitly allowed paths
Network	Unauthorized outbound connections
Process	Privilege escalation and dangerous syscalls
Inference	Model API calls rerouted to controlled backends only

⚠ WARNING

Security warning: Run NemoClaw in a clean environment or VM. Keep it isolated like a sandbox — do not expose host filesystem or network beyond what is explicitly allowed in policy. Never store personal credentials on the sandbox device.

CHAPTER 9 — Real-World Applications

Production Use Cases for Sovereign Edge AI

9.1 Smart Building / Hotel HVAC MAS

A multi-agent system on DGX Spark + Azure hybrid cloud manages a 500-room hotel's HVAC system: 4 chillers, 40 AHUs, 500 FCUs, with 40 fault detection rules and reinforcement learning for energy optimization. The DGX Spark runs all agents locally with zero latency, while Azure stores historical data and serves dashboards.

```
# Smart Hotel HVAC MAS — DGX Spark edge AI stack
# Stack: Ollama (Gemma 4 31B) + TimescaleDB + Neo4j + LiteLLM
from openai import OpenAI
hvac_agent = OpenAI(
base_url="http://localhost:11434/v1",
api_key="ollama"
)
def diagnose_fault(fault_code: str, sensor_data: dict) -> str:
resp = hvac_agent.chat.completions.create(
model="gemma4:31b",
messages=[
{"role": "system", "content":
"You are an expert HVAC fault diagnosis agent. "
"Analyse sensor data and recommend actions."},
{"role": "user", "content":
f"Fault: {fault_code}\nSensors: {sensor_data}"}
],
tools=[{
"type": "function",
"function": {
"name": "dispatch_maintenance",
"description": "Dispatch maintenance team to equipment",
"parameters": {
"type": "object",
"properties": {
"equipment": {"type": "string"},
"priority": {"type": "string",
"enum": ["low","medium","high","critical"]}
}
}
}
}]
)
```

```
return resp.choices[0].message
```

9.2 Robotics & VLA (Vision-Language-Action)

Gemma 4 multimodal models on Jetson Thor enable Voice-Vision-Action pipelines: the robot sees through a camera, hears voice commands, and acts in the physical world — all at sub-100ms latency with no cloud dependency.

- Drone navigation: Jetson Orin + Gemma 4 E4B + camera → understand "go to the red door" → navigate
- Factory inspection: Jetson Thor + 31B → identify defects from camera + generate inspection report
- Warehouse automation: Jetson Orin NX + LLM + TensorRT YOLO → pick-and-place with natural language commands
- Security patrol: Jetson AGX + Gemma 4 → detect anomalies, describe in natural language, alert via MQTT

9.3 Private Enterprise Assistant

```
# NemoClaw + DGX Spark + Telegram = private company assistant
# All data stays on-premise, accessible via phone from anywhere
# 1. Install NemoClaw (from Chapter 8)
nemoclaw onboard --fresh --gpu
# 2. Configure company knowledge base
# Point the assistant at internal docs folder:
nemoclaw configure --rag-path /data/company-docs \
--embedding-model nomic-embed-text
# 3. Telegram bot setup (for remote access)
nemoclaw configure --telegram-token YOUR_BOT_TOKEN
# 4. Result: any team member can ask:
# "What is our Q2 revenue target?" → searches internal docs → answers
# All processing on DGX Spark, never leaves the building
```

9.4 Sovereign Government & IoT Air-Gapped Deployment

Raspberry Pi 5 clusters run Gemma 4 E2B for air-gapped environments where absolutely no external connectivity is permitted: classified government networks, industrial control systems, medical devices.

- Air-gapped sovereign cluster: 4× Raspberry Pi 5 nodes, each running Gemma 4 E2B, load-balanced by LiteLLM — total cost ~$400, no cloud dependency
- Medical edge: Patient data summarization on-device, HIPAA/GDPR compliance by design
- Utility SCADA: AI anomaly detection on industrial sensors without internet exposure

• Embassy/consulate: Multilingual document translation + summarization with Gemma 4's 140+ language support

9.5 Mobile Learning & Health

Gemma 4's Agent Skills Gallery enables powerful on-device applications with no API costs:

Agent Skill	What It Does	Platform
Wikipedia Enrichment	Query Wikipedia to answer any encyclopedic question	Android/iOS
Video → Flashcards	Transform any video into study flashcards	Android/iOS
Sleep Mood Tracker	Summarize daily sleep + mood trends from speech input	Android/iOS
Photo → Music	Generate music suggestions matching photo mood	Android/iOS
Animal Calls App	Describe and play animal vocal calls from conversation	Android/iOS
Code Generator	Write and explain code entirely offline	Mac/Linux/Windows

CHAPTER 10 — Security, Troubleshooting & Best Practices

Running Sovereign AI Safely in Production

10.1 Security Best Practices

Risk	Mitigation
Prompt injection	Use NemoClaw OpenShell + input validation; never embed user input into tool names
Network exposure	Bind Ollama/vLLM to 127.0.0.1 or VPN only; use Tailscale for remote access
Model exfiltration	Self-encrypting NVMe on DGX Spark; disk encryption on all platforms
Data leakage via logs	Use --no-access-log in vLLM; set log level to WARN in production
Privilege escalation	Run all serving containers with --user $(id -u):$(id -g), no root
Supply chain attack	Verify model checksums from official HuggingFace repos; pin versions

10.2 Common Troubleshooting

Problem	Symptom	Fix
Ollama uses CPU on DGX Spark	<13 tok/s, GPU util = 0%	Upgrade to Ollama 0.24+; missing sm_121 CUDA libs
OOM error vLLM	CUDA out of memory	Reduce --gpu-memory-utilization to 0.82; use NVFP4 or Q4
Slow prefill Pi 5	<1 tok/s prefill	Use LiteRT-LM instead of llama.cpp for Pi 5
Jetson Gemma 4 slow	~2 tok/s on Jetson	Run JetPack update; upgrade Ollama; enable GPU: --n-gpu-layers 99
Tool calling broken	JSON parse errors	Use --tool-call-parser hermes in vLLM; update to Ollama 0.24+
Mac Flash Attention bug	Crash on Apple Silicon	Use OLLAMA_FLASH_ATTENTION=0 env var; fixed in latest Ollama
Context too slow	Throughput drops >20K tokens	Reduce ctx to 8K; Gemma 4 real-world limit ~20K on consumer GPU
High TTFT	>3s first token	Enable continuous batching in vLLM; reduce --max-model-len

10.3 Performance Optimization Checklist

1. Verify GPU is active: Run nvidia-smi during inference. GPU utilization should be 80–95%.
2. Choose right quantization: Q4_K_M for Ollama/llama.cpp. NVFP4 for vLLM on Blackwell.
3. Set context conservatively: Start at 8K tokens. Increase only if needed — memory grows linearly.
4. Use batch requests: vLLM continuous batching handles concurrency much better than Ollama.
5. Enable speculative decoding: Ollama 0.23.1+ on Mac, llama.cpp MTP on Linux (+40% on Gemma 4 26B).
6. Tune --shm-size: Set to 32–64 GB for vLLM Docker; default 64 MB causes silent OOM crashes.
7. Monitor with nvidia-smi dmon: Real-time GPU utilization, memory, and temperature.
8. Use MoE over Dense: Gemma 4 26B MoE is 6× faster than 31B Dense with only ~2% quality loss.

10.4 Monitoring Your Sovereign AI Deployment

```
# Real-time GPU monitoring
nvidia-smi dmon -s u,m,t # utilization, memory, temperature every second
# Check active vLLM metrics (Prometheus endpoint)
curl http://localhost:8000/metrics | grep vllm
# Ollama system info
ollama ps # models currently loaded in GPU memory
# Tailscale for secure remote SSH access
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up # creates private WireGuard tunnel
# Now SSH from anywhere: ssh user@100.x.x.x
# Benchmark your DGX Spark (community tool)
# pip install llama-benchy
# llama-benchy --model gemma4:26b --pp 2048 --tg 128
# Check LiteRT-LM device capabilities
litert-lm devices # lists all supported accelerators on this device
```

10.5 Resources & Next Steps

Resource	URL	What You'll Find
DGX Spark Playbooks	https://build.nvidia.com/spark	All official playbooks: vLLM, NeMo, NemoClaw, fine-tune
Google AI Edge	https://developers.google.com/edge	LiteRT-LM docs, AI Edge Gallery, device benchmarks
Gemma 4 Agentic Edge	https://developers.googleblog.com/..gemma-4/	Agent Skills guide, LiteRT-LM CLI, mobile deployment

NVIDIA NeMo Fine-tune	https://build.nvidia.com/spark/nemo-fine-tune	Step-by-step NeMo AutoModel on DGX Spark
NeMo Agent Toolkit	https://docs.nvidia.com/nemo/agent-toolkit/	Full agent framework docs, workflow examples
NemoClaw	https://build.nvidia.com/spark/nemoclaw	Secure sandbox setup + Telegram bot guide
Ollama Library	https://ollama.com/library	All available Gemma 4 and other model variants
DGX Spark Forum	https://forums.developer.nvidia.com/c/dgx-spark	Community benchmarks, LLM stacks, real configs
Jetson AI Lab	https://www.jetson-ai-lab.com	Jetson containers, VLA demos, LLM benchmarks
Google AI Edge Portal	https://developers.google.com/edge/ai-edge-portal	Benchmark 100+ real devices with LiteRT models
Gemma 4 on HuggingFace	https://huggingface.co/google/gemma-4-27b-it	Model cards, GGUF downloads, NVFP4 checkpoints

DEPLOY

You now have everything needed for a complete sovereign AI deployment. Start with Ollama on your current hardware, run gemma4:26b, and build from there. The same skills scale from a Raspberry Pi to a DGX Spark cluster.

APPENDIX — Quick Reference

Sovereign AI Edge — Quick Reference Card

One-Command Quickstart by Platform

```
# DGX Spark / Ubuntu PC (NVIDIA GPU)
curl -fsSL https://ollama.com/install.sh | sh && ollama run gemma4:26b
# Raspberry Pi 5
curl -fsSL https://ollama.com/install.sh | sh && ollama run gemma4:2b
# Mac (Apple Silicon)
brew install ollama && ollama run gemma4:26b
# Phone (Android)
# Install "Google AI Edge Gallery" from Play Store → Agent Skills
# Jetson Orin
# cd jetson-containers && ./run.sh $(./autotag ollama)
# Inside container: ollama run gemma4:4b
# LiteRT-LM (cross-platform)
pip install litert-lm && litert-lm run gemma4-e2b
```

Model Selection by VRAM

Available RAM/VRAM	Best Model	Quantization	Expected Speed
< 2 GB (phone)	Gemma 4 E2B	2-bit (LiteRT)	7–31 tok/s
3–4 GB (Jetson Nano)	Gemma 4 E4B	Q4 (LiteRT / GGUF)	12–20 tok/s
8 GB	Gemma 4 E4B or Llama 3.2 1B	Q4_K_M	20–54 tok/s
14–16 GB (Mac 16 GB)	Gemma 4 26B MoE	Q4_K_M	15–30 tok/s
24 GB (RTX 4090)	Gemma 4 31B Dense	Q4_K_M	~35 tok/s
64 GB (Mac M2 Ultra)	Gemma 4 31B Dense	Q8_0	~20 tok/s
128 GB (DGX Spark)	Gemma 4 31B Dense BF16	BF16/NVFP4	23–65 tok/s

Key Environment Variables

```
# Ollama
OLLAMA_HOST=0.0.0.0:11434 # bind to all interfaces (for network access)
```

```
OLLAMA_MODELS=/data/models # custom model storage path
OLLAMA_FLASH_ATTENTION=0 # disable Flash Attention on Mac (bug fix)
OLLAMA_NUM_PARALLEL=4 # concurrent request slots
# vLLM
VLLM_WORKER_MULTIPROC_METHOD=spawn # required for multi-GPU
CUDA_VISIBLE_DEVICES=0,1 # select specific GPUs
# LiteLLM
LITELLM_LOG=ERROR # reduce log verbosity in production
# HuggingFace cache
HF_HOME=/data/hf_cache # custom cache location (important for large models)
HUGGING_FACE_HUB_TOKEN=hf_... # for private/gated models like Gemma 4
```